

Portfolio App

Architecture & Code Flow Documentation

Flutter Web & Mobile Application

Developer: Vishnu

March 2026

Built with Flutter | Provider | Custom Animations

Table of Contents

1. Project Overview & Tech Stack
2. File Structure Map
3. App Startup Flow
4. Data Layer (Models & Data Source)
5. Theme System & State Management
6. Navigation Architecture
7. Section-by-Section Breakdown
 - 7.1 Hero Section
 - 7.2 About Section
 - 7.3 Skills Section
 - 7.4 Experience Section
 - 7.5 Projects Section
 - 7.6 Contact Section
8. Widget Library
 - 8.1 Navigation Bar
 - 8.2 Footer
 - 8.3 Chatbot
9. Animation System
 - 9.1 Game Loading Screen
 - 9.2 Parallax Background
 - 9.3 Particle System
 - 9.4 Matrix Rain
 - 9.5 Mouse Follower
 - 9.6 Card 3D & Glitch Text
 - 9.7 Section Reveal
10. Complete Data Flow Diagram
11. Function Reference Table

1. Project Overview & Tech Stack

This is a single-page Flutter portfolio application that showcases Vishnu's skills, projects, and experience as a Mobile Application Developer. The app features rich animations, a dark/light theme system, responsive design, and an AI-style chatbot.

Tech Stack

Framework	Flutter 3.43.0 (Dart 3.10.0)
State Mgmt	Provider 6.1.1 (ChangeNotifier pattern)
Animations	flutter_animate 4.5.0, rive 0.13.0, lottie 3.0.0, simple_animations 5.0.2
Icons	font_awesome_flutter 10.7.0, cupertino_icons 1.0.8
Navigation	Scroll-based (no router), url_launcher 6.2.4
Storage	shared_preferences 2.2.2 (theme persistence)
Text Effects	animated_text_kit 4.2.2
Utilities	visibility_detector 0.4.0, sprung 3.0.1
Platforms	Web, Android, iOS, macOS

2. File Structure Map

```

lib/
  main.dart                -- App entry point, HomePage scaffold
  data/
    portfolio_data.dart    -- All static content (bio, skills, projects, experience)
  models/
    project.dart           -- Project data model
    skill.dart             -- Skill & SkillCategory models
    message.dart           -- Chat Message model
  providers/
    theme_provider.dart    -- Dark/Light theme state (ChangeNotifier)
  theme/
    app_theme.dart         -- ThemeData definitions (colors, typography)
  services/
    chatbot_service.dart   -- Keyword-based chatbot response engine
  sections/
    hero/hero_section.dart -- Landing section with animated profile
    about/about_section.dart -- Bio, highlights, stats grid
    skills/skills_section.dart -- Skill categories with proficiency chips
    experience/experience_section.dart -- Timeline of work experience
    projects/projects_section.dart -- Featured project cards grid
    contact/contact_section.dart -- Contact info + form
  widgets/
    navigation/navigation_bar.dart -- Top nav bar with scroll-to-section
    footer/footer_section.dart -- Animated footer with wave effect
    chatbot/chatbot_widget.dart -- Floating chatbot overlay
  animations/
    game_loading_screen.dart -- Startup loading animation
    parallax_background.dart -- Multi-layer scroll parallax
    particle_system.dart -- Ambient floating particles
    matrix_rain.dart -- Digital rain + code snippets
    mouse_follower.dart -- Cursor trail effect
    card_3d.dart -- 3D hover card + GlitchText
    section_reveal.dart -- Scroll-triggered reveal animations

```

3. App Startup Flow

Boot Sequence

The app follows this startup sequence from launch to interactive state:

- 1. main() calls runApp(MyApp())
- 2. MyApp wraps the app in ChangeNotifierProvider<ThemeProvider>
- 3. ThemeProvider constructor loads saved theme from SharedPreferences
- 4. MaterialApp renders with the selected ThemeData (dark or light)
- 5. HomePage (StatefulWidget) is the root page
- 6. HomePage shows GameLoadingScreen overlay (isLoading = true)
- 7. GameLoadingScreen runs 3-second sequence:
 - - 6 text stages (500ms each): 'INITIALIZING...' through 'READY TO LAUNCH!'
 - - Progress bar animates 0% to 100%
 - - Rotating logo with 30 orbiting particles
- 8. onLoadingComplete() callback fires -> setState(isLoading = false)
- 9. Main content revealed: CustomScrollView with sections
- 10. Background layers activate: ParallaxBackground + ParticleSystem
- 11. MouseFollower overlay starts tracking cursor
- 12. ChatbotWidget floating button appears in bottom-right

Main Scaffold Layout (CustomScrollView)

After loading completes, the main page is a CustomScrollView with this structure:

- SliverAppBar (pinned) -> NavigationBar widget
- SliverList children:
 - [0] HeroSection -- Landing with profile, roles, CTA buttons
 - [1] AboutSection -- Bio, highlights, statistics
 - [2] SkillsSection -- Tech skill categories & proficiency
 - [3] ProjectsSection -- Featured project cards grid
 - [4] ContactSection -- Contact methods + form
 - [5] FooterSection -- Animated footer with wave effect
- Overlays (Stack):
 - - ParallaxBackground -- Multi-layer geometric parallax
 - - ParticleSystem -- Ambient floating particles/symbols
 - - MouseFollower -- Cursor trail effect
 - - ChatbotWidget -- Floating chat button + window

4. Data Layer

4.1 Static Data Source: PortfolioData

[lib/data/portfolio_data.dart](#)

All portfolio content lives in a single static class. No database or API -- everything is compiled into the app. This makes it fast and offline-capable.

Personal Info Fields

name	"Vishnu"
title	"Mobile Application Developer"
email	"vishnuvriez@gmail.com"
github	"https://github.com/VishnuViez"
linkedin	"https://www.linkedin.com/in/vishnuvriez/"
tagline	Crafting seamless mobile experiences...
bio	Multi-paragraph professional summary

Skill Categories (5)

Mobile Dev	Kotlin (95%), Java (90%), Flutter (85%), Android SDK (90%), Jetpack Compose (80%)
Architecture	MVVM (90%), MVP (85%), MVC (85%), Clean Arch (85%)
Networking	Retrofit (90%), REST APIs (90%), GraphQL (75%), WebSockets (80%)
Backend & AI	Python (80%), Flask (75%), ML (70%), TensorFlow (65%)
Tools	Git (90%), Firebase (85%), Room DB (85%), SQLite (80%), CI/CD (75%)

Projects (5)

E-Commerce App	Kotlin, MVVM, Retrofit, Room, Material Design
Fitness Tracker	Flutter, Dart, Provider, Charts, Animations
Social Dashboard	Kotlin, Jetpack Compose, GraphQL, WebSockets
Chat Application	Kotlin, WebSockets, MVVM, Room, Encryption
Weather Forecast	Flutter, REST API, Provider, Animations, Geolocation

Experience (3 positions)

CronJ IT Tech.	SDE-2, Feb 2023 - Dec 2024
Aroha Tech.	Android Dev, Dec 2021 - Nov 2022
Infount	Software Dev, Jun 2019 - Aug 2021

4.2 Data Models

Project Model

[lib/models/project.dart](#)

```
class Project {
  String title, description
  List<String> technologies, highlights
  String? imageUrl, githubUrl, liveUrl
}
```

Skill Models

[lib/models/skill.dart](#)

```
class Skill {  
  String name, category  
  double proficiency // 0.0 to 1.0  
}
```

```
class SkillCategory {  
  String name  
  List<Skill> skills  
}
```

Message Model

[lib/models/message.dart](#)

```
class Message {  
  String text  
  bool isUser // true=user, false=bot  
  DateTime timestamp  
}
```

5. Theme System & State Management

5.1 Theme Definitions

[lib/theme/app_theme.dart](#)

Two complete ThemeData configurations:

Dark Theme Colors

Primary	#60A5FA (Light Blue)
Secondary	#A78BFA (Light Purple)
Background	#0F172A (Dark Slate)
Surface	#1E293B (Slate)
Text	#F1F5F9 (Near White)

Light Theme Colors

Primary	#2563EB (Blue)
Secondary	#7C3AED (Purple)
Background	#F8FAFC (Light Gray)
Surface	#FFFFFF (White)
Text	#0F172A (Dark Slate)

5.2 Theme Provider (State Management)

[lib/providers/theme_provider.dart](#)

Uses Provider pattern with ChangeNotifier for reactive theme switching:

```
class ThemeProvider extends ChangeNotifier {
  bool _isDarkMode = false;

  ThemeProvider() { _loadThemePreference(); }

  _loadThemePreference() async {
    // Reads 'isDarkMode' from SharedPreferences
    // Calls notifyListeners() to update UI
  }

  toggleTheme() async {
    _isDarkMode = !_isDarkMode;
    // Saves to SharedPreferences
    notifyListeners(); // Triggers rebuild
  }
}
```

Flow: User taps theme toggle in NavigationBar -> toggleTheme() called -> SharedPreferences updated -> notifyListeners() -> MaterialApp rebuilds with new ThemeData -> All theme-aware widgets update colors automatically.

6. Navigation Architecture

The app uses scroll-based navigation instead of route-based navigation. All sections are stacked vertically in a single CustomScrollView. Clicking a nav item scrolls to the corresponding section.

Navigation Flow

- NavigationBar displays 5 items: Home, About, Skills, Projects, Contact
- Each section widget has a GlobalKey for position tracking
- User clicks nav item -> onSectionTap(index) callback fires
- HomePage._scrollToSection(index) is called
- Scrollable.ensureVisible() animates scroll to the section's GlobalKey context
- Animation: 800ms duration with easeInOutCubic curve
- NavigationBar updates _selectedIndex to highlight active item

NavigationBar Widget

[lib/widgets/navigation/navigation_bar.dart](#)

Key features:

- Pinned SliverAppBar (always visible at top)
- Desktop: Horizontal row of clickable NavItems + theme toggle
- Mobile: Hamburger icon -> modal bottom sheet with nav items
- Active item: Blue underline indicator with scaleX animation
- Hover effect: Color change with smooth transition
- Logo: Gradient box with code icon, fades in on load

7. Section-by-Section Breakdown

7.1 Hero Section

[lib/sections/hero/hero_section.dart](#)

The landing section -- first thing users see after loading screen.

Components

- Animated circular profile picture with gradient border ring
- Ring scales 1.0 to 1.1 continuously (breathing animation)
- 6 floating tech badges orbiting the profile (Flutter, Kotlin, Python, AI/ML, AWS, Docker)
- Badge positions calculated with cos/sin for circular orbit
- AnimatedTextKit cycles through 5 roles with typewriter effect
- Two CTA buttons: 'View My Work' (scrolls to projects) and 'Download Resume'
- 3 social buttons (GitHub, LinkedIn, Email) with hover scale animation

Key Functions

<code>build()</code>	Renders responsive layout (Row for desktop, Column for mobile)
<code>_SocialButton.build()</code>	Renders icon button with hover scale and color animation
<code>_FloatingBadge.build()</code>	Positions badge on circular orbit using trigonometry

7.2 About Section

[lib/sections/about/about_section.dart](#)

- Section title with animated horizontal line
- Professional bio from PortfolioData.bio
- 6 'What I Bring' highlights with emoji bullets
- Statistics grid: 50+ Projects, 5+ Years, 15+ Technologies, 100% Satisfaction
- VisibilityDetector triggers animations at 30% visibility

Key Functions

<code>build()</code>	VisibilityDetector wrapping responsive Row/Column layout
<code>_buildStats()</code>	Returns GridView with 4 stat cards (scale animation)

7.3 Skills Section

[lib/sections/skills/skills_section.dart](#)

- Iterates PortfolioData.skillCategories (5 categories)
- Each category: title + Wrap of skill chips
- Each chip shows: skill name + proficiency % + gradient background
- Staggered fadeIn + scale animations per chip

7.4 Experience Section

`lib/sections/experience/experience_section.dart`

- Timeline-style vertical layout with connecting lines
- 3 experience entries from PortfolioData.experience
- Each card: company name, position, duration badge, description, 5 achievements
- Left-aligned circular indicators with vertical lines between
- Cards fadeIn + slideX from right with 200ms staggered delay

7.5 Projects Section

`lib/sections/projects/projects_section.dart`

- Responsive GridView: 1 column mobile, 2 columns desktop
- 5 project cards from PortfolioData.projects
- Each card wrapped in Card3D for 3D mouse-tracking hover effect
- Card content: gradient icon, GlitchText title, description, highlights, tech tags, buttons
- 5 unique gradient color schemes rotate across cards
- Action buttons: GitHub (Code) and Live Demo links via url_launcher
- Shimmer overlay animation after card reveal

Key Functions

<code>_ProjectCard.build()</code>	Renders Card3D > Card > Stack with gradient background and content
<code>_launchUrl(url)</code>	Opens GitHub/Live URL via url_launcher package
<code>_ProjectsSectionState.build()</code>	LayoutBuilder determines grid columns from width

7.6 Contact Section

`lib/sections/contact/contact_section.dart`

- Left side: 'Let's work together!' heading + 3 contact method cards (Email, GitHub, LinkedIn)
- Right side: Contact form with Name, Email, Message fields
- Form validation: required fields, email format check
- Submit shows success SnackBar and resets form
- Contact methods open URLs via url_launcher on tap
- Hover effect on contact cards: background color change

Key Functions

<code>_sendMessage()</code>	Validates form with GlobalKey<FormState>, shows SnackBar
<code>_launchUrl(url)</code>	Opens contact URLs (email/github/linkedin)
<code>_buildContactInfo()</code>	Renders contact method cards with animations
<code>_buildContactForm()</code>	Renders validated form with text fields

8. Widget Library

8.1 NavigationBar

[lib/widgets/navigation/navigation_bar.dart](#)

Pinned app bar with responsive desktop/mobile navigation, theme toggle, and scroll-to-section.

build()	SliverAppBar with Row (desktop) or hamburger menu (mobile)
_showMobileMenu()	Opens ModalBottomSheet with nav items
_NavItem.build()	Clickable nav item with hover underline animation
onSectionTap(index)	Callback to HomePage._scrollToSection()

8.2 Footer Section

[lib/widgets/footer/footer_section.dart](#)

Animated footer with wave background, floating logo, tech badges, and glitch copyright.

3 AnimationControllers	Wave (4s loop), Float (3s reverse), Pulse (2s reverse)
_WavePainter	Custom sine-wave painting on animated canvas
Floating Logo	'<Vishnu />' with pulsing scale and glow shadow
Tech Badges	5 emoji tech icons in Wrap layout
Copyright Glitch	Red/blue offset text layers triggered at 95% pulse

8.3 Chatbot Widget

[lib/widgets/chatbot/chatbot_widget.dart](#)

Floating chat overlay with keyword-based AI responses about the portfolio.

Chat Flow

- 1. Floating button (bottom-right) with pulsing gradient animation
- 2. Tap opens chat window (380x600 desktop, fullscreen mobile)
- 3. Welcome message from ChatbotService shown initially
- 4. Quick suggestion chips shown when message count <= 1
- 5. User types message -> _sendMessage() called
- 6. User message added to list, typing indicator shown
- 7. 1-second delay simulates 'thinking'
- 8. ChatbotService.getResponse() matches keywords -> returns response
- 9. Bot message added, typing indicator removed, auto-scroll to bottom

Key Functions

_sendMessage(text)	Adds user msg, shows typing, fetches bot response after 1s delay
_scrollToBottom()	Scrolls ListView to end after new message
_MessageBubble	Styled message with icon, timestamp, user/bot alignment
_TypingIndicator	3 pulsing dots with staggered animation

ChatbotService Keywords

[lib/services/chatbot_service.dart](#)

hello/hi/hey	Greeting response
skills/technologies	Full skill category breakdown
projects/portfolio	5 project descriptions
experience/work	3 positions with achievements
contact/email/hire	Email, GitHub, LinkedIn links
kotlin/flutter/mvvm/ml	Specific tech deep-dives
about/who	Professional bio summary
education	Learning resources guidance
available/job	Availability and opportunity types
default	Fallback with topic suggestions

9. Animation System

The app uses 5 animation approaches: `flutter_animate` (declarative chains), `AnimationController` (low-level), `CustomPaint` (geometry), `Transform/Matrix4` (3D), and implicit `AnimatedBuilder` (reactive).

9.1 Game Loading Screen

[lib/widgets/animations/game_loading_screen.dart](#)

Rotating Logo	Central circle with 8 orbiting dots, 2s rotation cycle, pulsing scale
Particle Field	30 particles with orbital motion around center
Loading Text	6 stages at 500ms intervals: INITIALIZING -> READY TO LAUNCH
Progress Bar	Animated gradient fill from 0-100%, 3s total duration
onLoadingComplete	Callback after 3s triggers main content reveal

9.2 Parallax Background

[lib/widgets/animations/parallax_background.dart](#)

Layer 1 (10%)	Slow circles pattern - subtle depth
Layer 2 (30%)	Hexagonal grid pattern
Layer 3 (50%)	Grid lines that shift with scroll
Layer 4 (Orbs)	5 floating orbs with radial gradients and orbital motion
Layer 5	Child content (foreground)
Gradient BG	20-second animated color cycle using sin/cos lerp
Scroll tracking	<code>ScrollController</code> listener updates parallax offset

9.3 Particle System

[lib/widgets/animations/particle_system.dart](#)

Particle class	Position, velocity, size, opacity, color, rotation, symbol
Initialization	Random particles across screen with velocity vectors
Update loop	60fps: move by velocity, wrap at edges, rotate
Code symbols	20 chars: { } < > / () [] ; = + - * & ? ! # @
Painter	Draws circles (default) or rotated text (symbol mode)
Usage	<code>IgnorePointer</code> overlay on main page for ambient effect

9.4 Matrix Rain

[lib/widgets/animations/matrix_rain.dart](#)

MatrixRain	Digital rain with 50 columns of falling characters
Character set	40 chars: 0-9, A-Z, and code symbols
Column mgmt	Each column: position, speed, char list, length; resets at bottom
Mutation	5% probability of random character change per frame
Rendering	Head char is bright white, trail fades with decreasing opacity
CodeSnippets	Bouncing code text blocks that bounce off screen edges

9.5 Mouse Follower

[lib/widgets/animations/mouse_follower.dart](#)

Trail Points	Max 20 points tracking cursor, each with position/opacity/size/color
Fade behavior	Opacity * 0.95 and Size * 0.98 each frame
Color cycling	HSL hue cycles 0-360 degrees over 6 seconds
Painter	Draws glow (30% opacity, 10px blur) + core (full opacity, 3px blur)
Usage	IgnorePointer overlay so trail doesn't block clicks

9.6 Card 3D & Glitch Text

[lib/widgets/animations/card_3d.dart](#)

Card3D	Wraps any widget with mouse-tracking 3D tilt effect
3D Math	$\text{rotationX} = -\text{mouseY} * \text{maxRotation}$, $\text{rotationY} = \text{mouseX} * \text{maxRotation}$
Matrix4	Perspective (0.001) + rotateX + rotateY transforms
Hover effects	Shadow shift toward mouse, scale 1.0->1.05, shine gradient overlay
_ShinePainter	Radial gradient at mouse position with animated opacity
GlitchText	Random red/blue offset text layers triggered at 80%+ animation
HolographicCard	Iridescent color-shifting effect following mouse position

9.7 Section Reveal

[lib/widgets/animations/section_reveal.dart](#)

Scroll-triggered reveal animations with 8 styles:

slideUp	Translate from bottom + fadeIn
slideLeft	Translate from right + fadeIn
slideRight	Translate from left + fadeIn
scale	Scale 0.8->1.0 + fadeIn
rotate	Rotate 17 deg -> 0 deg + fadeIn
flip	3D X-axis flip 0->-90 deg + fadeIn
expand	ClipRect heightFactor animation
blur	Simple opacity fade

Also includes GradientText (animated shimmer mask) and PulsatingBorder widgets.

10. Complete Data Flow Diagram

How data and events flow through the application:

Theme Flow

```
User taps theme toggle (NavigationBar)
-> ThemeProvider.toggleTheme()
-> SharedPreferences.setBool('isDarkMode', value)
-> notifyListeners()
-> Consumer<ThemeProvider> in MyApp rebuilds
-> MaterialApp gets new ThemeData
-> All Theme.of(context) calls updated
```

Navigation Flow

```
User clicks nav item (NavigationBar._NavItem)
-> onSectionTap(index) callback
-> HomePage._scrollToSection(index)
-> Scrollable.ensureVisible(sectionKeys[index].currentContext)
-> 800ms smooth scroll animation
-> NavigationBar._selectedIndex updated
```

Chatbot Flow

```
User types message in chat input
-> _sendMessage(text)
-> Message(text, isUser:true) added to _messages list
-> _isTyping = true (shows TypingIndicator)
-> Future.delayed(1 second)
-> ChatbotService.getResponse(text)
    -> Keyword matching against 11 patterns
    -> Returns template string with PortfolioData injected
-> Message(response, isUser:false) added
-> _isTyping = false
-> _scrollToBottom()
```

Section Visibility Flow

```
User scrolls page
-> VisibilityDetector.onVisibilityChanged fires
-> If visibleFraction > 0.3 (30% visible)
-> setState(() => _isVisible = true)
-> flutter_animate targets switch from 0 to 1
-> Animations play (fadeIn, slideX/Y, scale, etc.)
-> One-time trigger (won't replay on re-scroll)
```

Project Card Interaction Flow

```
User hovers over project card
-> Card3D tracks mouse position via MouseRegion
-> Matrix4 transform: perspective + rotateX/Y calculated
-> Shadow shifts, scale increases, shine overlay appears
```

User clicks 'Code' or 'Live' button

```
-> _launchUrl(url) called
-> url_launcher opens URL in browser
```


11. Function Reference Table

Complete index of key functions across all files:

main.dart

main()	Entry point, calls runApp(MyApp())
MyApp.build()	ChangeNotifierProvider + MaterialApp setup
HomePage.build()	Loading screen overlay + CustomScrollView scaffold
_onLoadingComplete()	Hides loading screen, reveals main content
_scrollToSection(idx)	Smooth scroll to section via GlobalKey

portfolio_data.dart

PortfolioData.*	Static fields: name, title, bio, email, github, linkedin
.skillCategories	List<SkillCategory> with 5 categories, 23 total skills
.projects	List<Project> with 5 featured projects
.experience	List of 3 work experience entries

theme_provider.dart

ThemeProvider()	Constructor, loads saved theme preference
_loadThemePref()	Reads SharedPreferences, sets _isDarkMode
toggleTheme()	Flips dark/light, saves pref, notifyListeners()
isDarkMode	Getter for current theme state

chatbot_service.dart

getResponse(msg)	Keyword-matches user message, returns response string
_containsAny()	Helper: checks if text contains any keyword from list
getSuggestions()	Returns 6 quick-action suggestion strings

Section Widgets (common pattern)

build()	VisibilityDetector > Container > responsive layout
_isVisible state	Triggers animation targets when section scrolled into view
isMobile check	MediaQuery.of(context).size.width < 768
.animate()	flutter_animate chain: fadeIn + slide/scale with delay

Animation Widgets

GameLoading.init	3-second sequence: 6 stages, progress bar, onComplete callback
ParallaxBG.scroll	ScrollController listener updates 5-layer parallax offsets
ParticleSystem.tick	60fps update: move particles, wrap edges, rotate symbols
MatrixRain.tick	Update column positions, mutate chars (5%), reset at bottom
MouseFollower.move	Add trail point, decay opacity*0.95, decay size*0.98
Card3D.hover	Calculate rotateX/Y from mouse, apply Matrix4 perspective
SectionReveal.vis	VisibilityDetector at 20% triggers chosen reveal style

GlitchText.tick

Random red/blue offset layers at >80% animation progress